

PROGRAMMAZIONE DI RETE IN JAVA



DEFINIZIONE SOCKET TCP



Nel linguaggio comune
SOCKET = PRESA DI CORRENTE

Un socket può essere pensato come un canale tramite il quale i processi che sono messi in comunicazione possono inviare e ricevere dati. Si tratta quindi di un canale bidirezionale.

Porte



I protocolli coinvolti nell'implementazione dei Socket possono essere:

TCP (Transfer Control Protocol)

UDP (User Datagram Protocol)

Un socket TCP è un estremo (end-point) di una connessione TCP tra due processi.

Un socket è caratterizzato da indirizzo IP e numero di porta.

Le porte



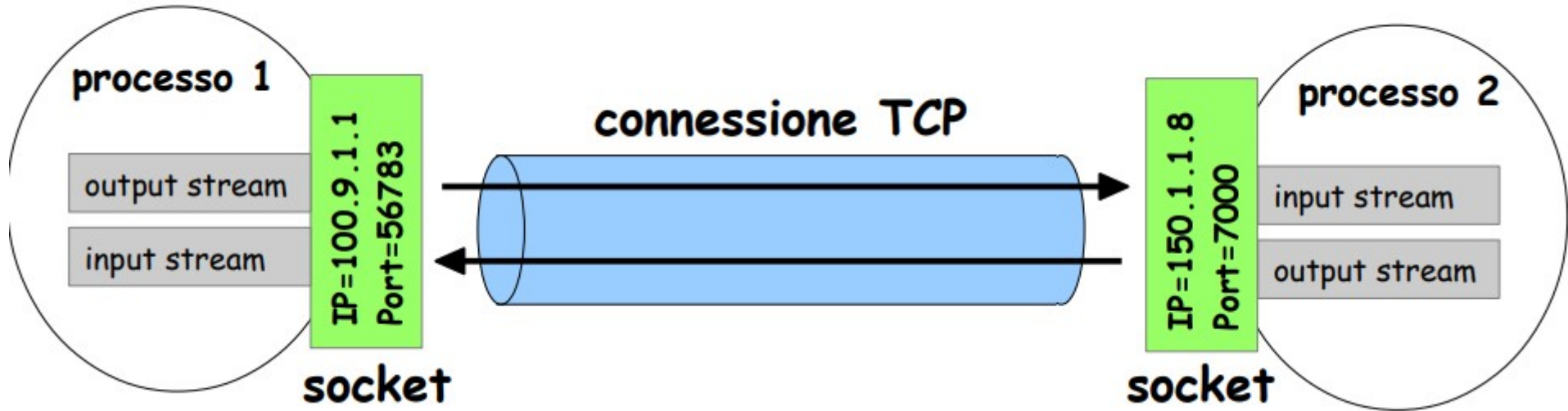
I numeri di porta sono numeri a 16 bit il cui valore può variare tra **0 e 65535**.

Non tutti questi valori possono essere utilizzati.

- I numeri di porta compresi tra **0 e 1023 sono riservati** a particolari servizi (telnet, ftp, SMTP, POP3 e HTTP e possono essere utilizzati soltanto in caso di interfacciamento con tali servizi)
- Le porte utilizzabili dagli utenti della rete sono quelle definite **Registered Ports**, i cui valori variano tra **1024 e 49151**.
- Sui numeri di porta compresi tra **49152 e 65535** non viene applicato alcun controllo (sono definite **private ports**).

Sia il client sia il server devono stabilire la porta da utilizzare per la comunicazione, altrimenti non riusciranno a comunicare (anche se l'indirizzo IP è corretto).

CONNESSIONE TCP



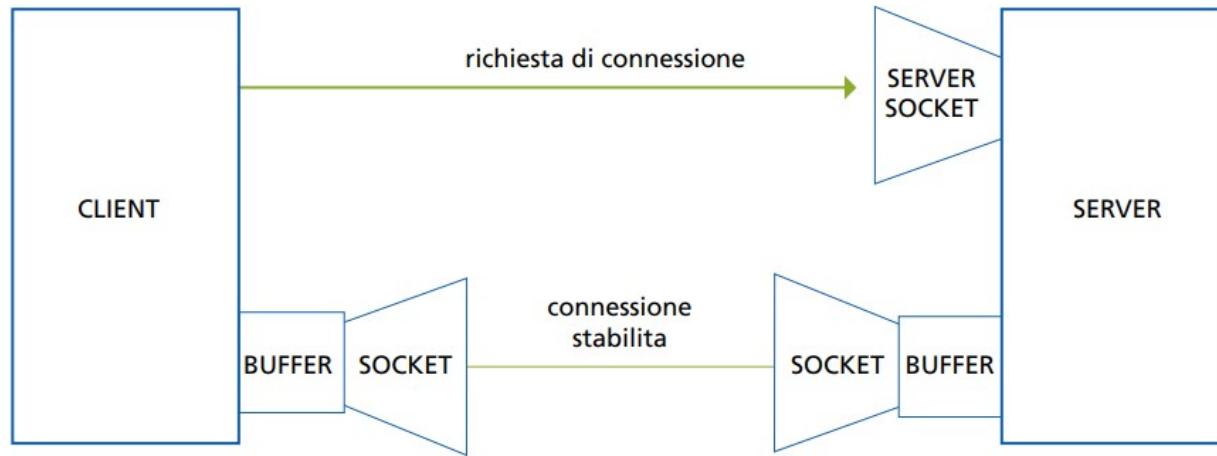
Una connessione TCP è un canale di comunicazione bidirezionale che consente a due processi di scambiare dati in modo affidabile tramite la rete.

Ognuno dei due estremi può leggere e scrivere dati sul canale.

Una connessione TCP è identificata da **4 valori**:

IP/porta del primo socket e IP/porta del secondo socket.

Connessione TCP



Il programma **server** crea un socket mediante l'apposita classe **ServerSocket** chiamando il **costruttore `ServerSocket(port)`**, che specifica nel parametro `port` il **numero della porta** sulla quale il server si metterà in ascolto delle richieste da parte dei client.

I programmi **client** creano la connessione con il socket mediante la classe **Socket** chiamando il **costruttore `Socket(serverAddress, port)`**, che specifica nel parametro `serverAddress` l'**indirizzo IP del server** e nel parametro `port` lo **stesso numero di porta usato dal server**.

Il **server**, ogni volta che riceve una richiesta di connessione da un client, **utilizza il metodo `accept()` della classe `ServerSocket` per creare un socket che gestisca la connessione** con quel client inviando quanto richiesto. Viene inoltre utilizzata la classe **`InetAddress` per gestire l'indirizzo IP**.

Socket in Java



Il package Java che fornisce supporto per l'implementazione dei Socket è **java.net**.

Le classi **Socket** e **ServerSocket** sono le due classi necessarie per l'implementazione di una comunicazione client-server basata sui socket.

Socket = si occupa della gestione del client che effettua la chiamata

ServerSocket = fornisce le funzionalità necessarie a creare un server che stia in ascolto su una particolare porta

Ip del dispositivo: **InetAddress.getLocalHost();**

Le classi in Java



Per stabilire una connessione sono necessari i seguenti passaggi:

- **Il server** sceglie una **porta** su cui mettersi in ascolto. Quando il client tenterà di aprire una connessione, il server richiamerà il **metodo accept()** della **classe ServerSocket** ottenendo un riferimento al canale di comunicazione instaurato con il client.
- **Il client** apre una connessione con il server con un oggetto di tipo **Socket** a cui verranno passati due parametri: **l'indirizzo IP del server e la porta su cui il server stesso è in ascolto**.

Una volta instaurata la connessione, server e client possono comunicare tramite gli stream (classi **InputStream** e **OutputStream**).

ESEMPIO



Vogliamo realizzare una semplice comunicazione tramite **TCP**:

- Il server ha indirizzo **IP 192.168.0.10** ed è in attesa di ricevere connessioni TCP sulla **porta 7000**
- il client instaura una connessione TCP con il server e rimane in attesa di ricevere dei dati
- il server, appena la connessione è instaurata, invia al client una stringa, poi chiude la connessione e termina.
- il client mostra a video la stringa ricevuta e termina.

Server

Il server richiede al S.O di poter ascoltare la porta TCP 7000



```
public class Server {  
    public static void main(String[] args) throws Exception {  
        ServerSocket servSocket = new ServerSocket(port: 7000);  
        System.out.println(x: "Server avviato");  
        Socket cs = servSocket.accept();  
        System.out.println("connessione da: " + cs.getRemoteSocketAddress());  
        PrintWriter printWriter =  
            new PrintWriter(out: cs.getOutputStream(), autoFlush: true);  
        printWriter.println(x: "ciao");  
        cs.close();  
    }  
}
```

server bloccato finchè non
riceve una richiesta di
connessione da un client

flush automatico dopo ogni scrittura

Client



IP e porta del server

```
public class Client {  
    public static void main(String[] args) throws Exception {  
        Socket s = new Socket( host:"192.168.0.10" , port:7000 );  
        BufferedReader buffRdr = new BufferedReader(  
            new InputStreamReader( in:s.getInputStream() ) );  
        String messaggio = buffRdr.readLine();  
        System.out.print( s:messaggio);  
    }  
}
```

il client si **BLOCCA** finchè
non riceve una riga completa
(ricezione del carattere
'\n' oppure '\r' oppure
della combinazione '\r\n')
OPPURE la connessione
viene chiusa

Server



Per creare un **Server** in ascolto è necessario:

- importare i **package java.io e java.net** per utilizzare le classi relative allo streaming ed alle Socket;
- intercettare le eccezione (**IOException** che deve essere gestita)
- creare un oggetto **ServerSocket** utilizzando il costruttore **ServerSocket(int port)** che accetta come parametro la porta del Server in ascolto per il servizio da attivare.
- richiamare il metodo **accept()** sull'oggetto **ServerSocket** per poter accettare le chiamate da parte del Client su un oggetto **Socket** .
- creare due oggetti **InputStreamReader e OutputStreamWriter** per le richieste del Client (stream in input e in output per realizzazione la comunicazione con il Client)
- richiamare il metodo **close()** sugli oggetti utilizzati per lo **streaming** al termine della comunicazione, per chiudere gli oggetti utilizzati
- richiamare il metodo **close()** sull'oggetto **Socket** per chiudere la connessione con il Client quando esso farà una richiesta di chiusura socket.close()

//Importo i package necessari

import java.net.*;

import java.io.*;

public class TCPServer {

public static void main (String[] args) throws Exception {

ServerSocket serverSocket = new ServerSocket(port:1234);

//Informazioni sul Server in ascolto

InetAddress indirizzo = serverSocket.getInetAddress();

String server = indirizzo.getHostAddress();

int port = serverSocket.getLocalPort();

System.out.println("In ascolto Server: " + server

+ " porta: " + port);

while (true) {

System.out.println(x:"In attesa di chiamate dai Client... ");

Socket socket = serverSocket.accept();

//Informazioni sul Client che ha effettuato la chiamata

InetAddress address = socket.getInetAddress();

String client = address.getHostName();

int porta = socket.getPort();

System.out.println("In chiamata Client: " + client + " porta

//Stream utilizzato per la comunicazione via socket

BufferedReader in = new BufferedReader(

new InputStreamReader(in:socket.getInputStream()));

creare due oggetti **InputStreamReader** e **OutputStreamWriter** per le richieste del Client (stream in input e in output per realizzazione la comunicazione con il Client)

creare un oggetto **ServerSocket** utilizzando il costruttore **ServerSocket(int port)**

richiamare il metodo **accept()** sull'oggetto **ServerSocket** per poter accettare le chiamate da parte del Client su un oggetto **Socket**.

```
BufferedWriter os = new BufferedWriter
(new OutputStreamWriter( out:socket.getOutputStream() ) );
while (true) {
    String userInput = in.readLine();
    if (userInput.equals( anObject:"QUIT"))
        break;
    os.write(userInput + '\n');
    System.out.println("Il Client ha scritto: " + userInput);
}
//chiusura della comunicazione con il Client
os.close();
in.close();
System.out.println("Chiusura chiamata Client: " + client + "su porta:"
socket.close());
```

Server



Per il Server:

Per ottenere l'indirizzo IP e il numero di porta del server si possono utilizzare:

- il metodo **getInetAddress()** della classe `InetAddress`;
- il metodo **getPort()** della classe `Socket`.

Client



Per creare un **Client** che effettui una chiamata ad un Server bisogna:

- importare i **package java.io, java.net** per poter utilizzare le classi relative allo Stream ed alle Socket
- creare un oggetto **Socket** e specificare l'**indirizzo IP ed il numero di porta in ascolto sul Server**.
- creare due oggetti **InputStreamReader e OutputStreamWriter** per le richieste del Client (stream in input e in output per realizzazione la comunicazione con il Client)
- richiamare il metodo **close()** sugli oggetti utilizzati per lo **streaming** al termine della comunicazione, per chiudere gli oggetti utilizzati
- richiamare il metodo **close()** sull'oggetto **Socket** per chiudere la connessione con il Client quando esso farà una richiesta di chiusura `socket.close()`

Per il **client**:

L'indirizzo IP viene ricavato, mentre la porta viene assegnata dinamicamente in base alla disponibilità. Se si vuole conoscere l'indirizzo IP e la porta del Client, è possibile utilizzare i metodi:

- **getLocalAddress()** della classe **InetAddress**;
- **getLocalPort()** della classe **Socket**.

Client



```
//Importo i package necessari
```

```
import java.net.*;
```

```
import java.io.*;
```

```
public class TCPCClient {
```

```
    public static void main (String[] args) throws Exception {
```

```
        //Connessione della Socket con il Server
```

```
        Socket socket = new Socket( host:"localhost", port:1234);
```

```
        //Stream da passare al Socket
```

```
        BufferedReader in = new BufferedReader(
```

```
            new InputStreamReader( in:socket.getInputStream() ) );
```

```
        BufferedWriter os = new BufferedWriter
```

```
            (new OutputStreamWriter( out:socket.getOutputStream() ) );
```

```
        BufferedReader stdIn = new BufferedReader
```

```
            (new InputStreamReader( in:System.in));
```

```
        System.out.print( s:"Per disconnettersi dal Server scrivere: QUIT\n");
```

```
        //Ciclo infinito per inserimento testo del Client
```

```
        while (true) {
```

```
            System.out.print( s:"Inserisci: ");
```

```
            String userInput = stdIn.readLine();
```

```
            if (userInput.equals( anObject:"QUIT\n"))
```

```
                break;
```

```
            os.write(userInput + '\n');
```

```
            os.flush();
```

```
            System.out.println("Hai digitato: " + in.readLine());
```

importare i **package java.io**, **java.net** per poter utilizzare le classi relative allo Stream ed alle Socket

creare un oggetto **Socket** e specificare l'**indirizzo IP** ed il **numero di porta in ascolto sul Server**.

richiamare il metodo **close()** sugli oggetti utilizzati per lo **streaming** e il **socket**

```
//Chiusura dello Stream e del Socket  
os.close();  
in.close();  
socket.close();  
}
```


Esercizio



Scrivi un programma Java che abbia un client e un server.

Il client genera 100 numeri casuali tra 0 e 20. Il server riceve i 100 numeri casuali e calcola la media. La media viene stampata su un file che chiamate output.txt. Scrivere un main di prova che chieda il servizio al server per 5 volte.

Server multi-thread



Il limite dell'esempio precedente è il fatto che il server riesce ad accettare le chiamate dei client in modo sequenziale, ovvero un solo client alla volta.

Se si utilizzano i thread si può far in modo che il server sia sempre in grado di accettare una chiamata indipendentemente dal fatto che in quel momento ci siano connessi altri client.

```
public class TCPParallelServer {  
    public static void main (String[] args) throws Exception {  
        ServerSocket serverSocket = new ServerSocket (port:1234);  
        //Ciclo infinito di ascolto dei Client  
        while(true) {  
            System.out.println( x:"In attesa di chiamate dai Client... ");  
            Socket socket = serverSocket.accept();  
            System.out.println("Ho ricevuto una chiamata di apertura da:\n"  
                               + socket);  
            ServerThread serverThread = new ServerThread(socket);  
            serverThread.start();  
        }  
    }  
}
```

richiamare il metodo **accept()** sull'oggetto **ServerSocket** per poter accettare le chiamate da parte del Client su un oggetto **Socket**.

Quando arriva una chiamata dal Client si avvia un thread passando la socket come parametro.

Classe ServerThread



```
class ServerThread extends Thread {
    private Socket socket;
    public ServerThread (Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            BufferedReader is = new BufferedReader(
                new InputStreamReader( in:socket.getInputStream()));
            BufferedWriter os = new BufferedWriter
                (new OutputStreamWriter( out:socket.getOutputStream()));
            while(true) {
                String userInput = is.readLine();
                if (userInput == null || userInput.equals( anObject:"QUIT"))
                    break;
                os.write(userInput + '\n');

                os.flush();
                System.out.println("Il Client ha scritto: " + userInput);
            }
            os.close();
            is.close();
            System.out.println("Ho ricevuto una chiamata di chiusura da:\n" +
                               socket + "\n");
            socket.close();
        }
        catch (IOException e) {
            System.out.println("IOException: " + e);
        }
    }
}
```

crea due oggetti **InputStreamReader** e **OutputStreamWriter** per le richieste del Client (stream in input e in output per realizzazione la comunicazione con il Client)

In un ciclo infinito, legge una riga e la rimanda al Client sull'OutputStreamWriter

Quando esce dal ciclo richiama il metodo **close()** sugli oggetti utilizzati per lo **streaming** e il **socket**

Tutto all'interno di un try catch

Esercizio



Scrivi un programma Java che abbia un client e un server. Il server offre il servizio di una semplice calcolatrice in grado di svolgere le quattro operazioni fondamentali.

Il client per ottenere il servizio dal server dovrà inviare i due operandi e l'operatore e il server fornirà il calcolo.

Provare a renderlo multi-thread

Classi principali



Classe	Descrizione
InetAddress	Rappresenta un indirizzo IP (indifferentemente versioni 4 a 32 bit o 6 a 128 bit) e/o un nome di dominio che risolve in modo trasparente utilizzando un servizio DNS; le classi derivate <code>Inet4Address</code> e <code>Inet6Address</code> rappresentano in modo specifico indirizzi IP versione 4 o 6.
Socket	Rappresenta il socket client di una connessione TCP; la classe derivata <code>SSLSocket</code> consente di usare le funzionalità del protocollo crittografico TLS (<i>Transport Layer Security</i>).
ServerSocket	Rappresenta il socket di ascolto di un server TCP; la classe derivata <code>SSLServerSocket</code> consente di usare le funzionalità del protocollo crittografico TLS.
DatagramSocket	Rappresenta il socket che impiega il protocollo UDP.
DatagramPacket	Rappresenta un <i>datagram</i> UDP ricevuto o da trasmettere: oltre ai dati comprende l'indirizzo IP mittente o di destinazione e il numero di porta UDP mittente o di destinazione.
URL	Rappresenta una qualsiasi risorsa presente nel web mediante il suo <i>Uniform Resource Locator</i> .

Esercizio in preparazione alla verifica del 13/12



Realizza un sistema in cui il server sia il banditore di un'asta: accoglie le offerte e comunica se sono accettabili o meno, comunica a richiesta la migliore offerta corrente; i client sono i partecipanti all'asta che possono richiedere quale sia l'offerta migliore (e di chi) e possono effettuare rilanci se il loro budget a disposizione lo consente.

Il client:

- ha un budget disponibile
- può chiedere al server l'attuale offerta migliore
- può rilanciare l'offerta (se il budget è sufficiente)
- può uscire dall'asta

Il server è multithread e riceve le varie richieste dai client:

- può inviare la migliore offerta attiva al client che l'ha richiesta
- può ricevere l'offerta dal client (in questo caso la confronta con l'offerta migliore e nel caso sia maggiore aggiorna l'offerta migliore)